Main Methods in String, StringBuffer, and StringBuilder in Java

This document explains the main methods of the `String`, `StringBuffer`, and `StringBuilder` classes in Java, including their syntax, purpose, and examples. These classes are used for string manipulation but differ in immutability and thread-safety.

1. String

The `String` class is **immutable**, meaning its value cannot be changed after creation. It is thread-safe and ideal for constant strings.

Main Methods

Method	Description	Syntax/Example
`length()`	Returns the length of the string.	`int len = str.length();` `"Hello".length(); // 5`
`charAt(int index)`	Returns the character at the specified index.	`char c = str.charAt(2);` `"Hello".charAt(2); // 'l'`
`substring(int beginIndex, int endIndex)`	Returns a substring from `beginIndex` to `endIndex-1`.	`String sub = str.substring(1, 4);` `"Hello".substring(1, 4); // "ell"`
`concat(String str)`	Concatenates the specified string to the end.	`String newStr = str.concat(" World");` `"Hello".concat(" World"); // "Hello World"`
`toUpperCase()` / `toLowerCase()`	Converts string to upper/lowercase.	`str.toUpperCase();` `"Hello".toUpperCase(); // "HELLO"`
`indexOf(String str)`	Returns the index of the first occurrence of `str`.	`int index = str.indexOf("l");` `"Hello".indexOf("l"); // 2`
`replace(CharSequence target, CharSequence replacement)`	Replaces all occurrences of `target` with `replacement`.	`String newStr = str.replace("l", "p");` `"Hello".replace("l", "p"); // "Heppo"`
`trim()`	Removes leading and trailing whitespace.	`String trimmed = str.trim();` `" Hello ".trim(); // "Hello"`
`equals(Object obj)`	Compares string content for equality.	`boolean eq = str.equals("Hello");` `"Hello".equals("hello"); // false`

`split(String regex)`	Splits string into an array based on `regex`.	`String[] arr = str.split(" ");`
		`"Hello World".split(" ");`
		`// ["Hello", "World"]`

Note: Since `String` is immutable, methods like `concat`, `replace`, etc., return a new `String` object.

2. StringBuffer

The `StringBuffer` class is **mutable** and **thread-safe** (synchronized methods), suitable for multi-threaded environments.

Main Methods

Method	Description	Syntax/Example
`append(String str)`	Appends the specified string to the buffer.	`buffer.append(" World");` `StringBuffer sb = new StringBuffer("Hello"); sb.append(" World"); // "Hello World"`
`insert(int offset, String str)`	Inserts `str` at the specified `offset`.	`buffer.insert(5, " Java");` `sb.insert(5, " Java"); // "Hello Java World"`
`delete(int start, int end)`	Deletes characters from `start` to `end-1`.	`buffer.delete(5, 10);` `sb.delete(5, 10); // "Hello World"`
`replace(int start, int end, String str)`	Replaces characters from `start` to `end-1` with `str`.	`buffer.replace(6, 11, "Java");` `sb.replace(6, 11, "Java"); // "Hello Java"`
`reverse()`	Reverses the sequence of characters.	`buffer.reverse();` `sb.reverse(); // "avaJ olleH"`
`length()`	Returns the length of the buffer.	`int len = buffer.length();` `sb.length(); // 10`
`capacity()`	Returns the current capacity of the buffer.	`int cap = buffer.capacity();` `sb.capacity(); // 16 (default) or more`
`setLength(int newLength)`	Sets the length of the buffer (truncates or pads with nulls).	`buffer.setLength(5);` `sb.setLength(5); // "Hello"`
`charAt(int index)`	Returns the character at the specified index.	`char c = buffer.charAt(2);` `sb.charAt(2); // 'l'`
`toString()`	Converts the buffer to a `String`.	`String str = buffer.toString();` `sb.toString(); // "Hello"`

Note: `StringBuffer` modifies the same object, so no new object is created during operations like `append` or `delete`.

3. StringBuilder

The `StringBuilder` class is **mutable** and **not thread-safe** (unsynchronized methods), offering better performance for single-threaded applications.

Main Methods

The methods of `StringBuilder` are identical to `StringBuffer` in functionality and syntax, but they are not synchronized.

Method	Description	Syntax/Example
`append(String str)`	Appends the specified string.	`builder.append(" World");` `StringBuilder sb = new StringBuilder("Hello"); sb.append(" World");` `// "Hello World"`
`insert(int offset, String str)`	Inserts `str` at `offset`.	`builder.insert(5, " Java");` `sb.insert(5, " Java"); // "Hello Java World"`
`delete(int start, int end)`	Deletes characters from `start` to `end-1`.	`builder.delete(5, 10);` `sb.delete(5, 10); // "Hello World"`
`replace(int start, int end, String str)`	Replaces characters from `start` to `end-1` with `str`.	`builder.replace(6, 11, "Java");` `sb.replace(6, 11, "Java"); // "Hello Java"`
`reverse()`	Reverses the sequence of characters.	`builder.reverse();` `sb.reverse(); // "avaJ olleH"`
`length()`	Returns the length of the builder.	`int len = builder.length();` `sb.length(); // 10`
`capacity()`	Returns the current capacity.	`int cap = builder.capacity();` `sb.capacity(); // 16 (default) or more`
`setLength(int newLength)`	Sets the length of the builder.	`builder.setLength(5);` `sb.setLength(5); // "Hello"`
`charAt(int index)`	Returns the character at the specified index.	`char c = builder.charAt(2);` `sb.charAt(2); // 'l'`
`toString()`	Converts the builder to a `String`.	`String str = builder.toString();` `sb.toString(); // "Hello"`

Key Differences

Feature	`String`	`StringBuffer`	`StringBuilder`
Mutability	Immutable	Mutable	Mutable
Thread-Safety	Thread-safe	Thread-safe (synchronized)	Not thread-safe
Performance	Slowest (creates new objects)	Slower (due to synchronization)	Fastest (no synchronization)

Use Case	Constant strings	Multi-threaded string manipulation	Single-threaded string manipulation

Example Code

Below is a Java program demonstrating the usage of these classes:

```java
public class StringDemo {
    public static void main(String[] args) {
        // String
        String str = "Hello";
        System.out.println(str.concat(" World")); // Hello World
        System.out.println(str.toUpperCase()); // HELLO
        System.out.println(str); // Hello (original unchanged)

        // StringBuffer
        StringBuffer buffer = new StringBuffer("Hello");
        buffer.append(" World");
        buffer.insert(5, " Java");
        System.out.println(buffer); // Hello Java World
        buffer.reverse();
        System.out.println(buffer); // dlroW avaJ olleH

        // StringBuilder
        StringBuilder builder = new StringBuilder("Hello");
        builder.append(" World");
        builder.delete(5, 10);
        System.out.println(builder); // Hello World
        builder.replace(6, 11, "Java");
        System.out.println(builder); // Hello Java
    }
}
```

This Markdown file can be downloaded and used as a reference for understanding `String`, `StringBuffer`, and `StringBuilder` methods in Java.